

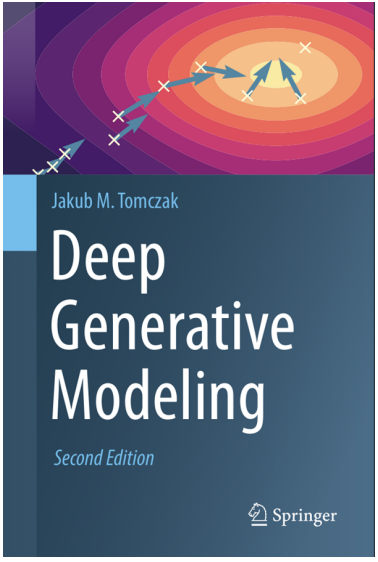
MACQUARIE University
SYDNEY·AUSTRALIA

Advanced Machine Learning - COMP8221

Week 3: Deep Generative Models - Part III

Dr. Qiongkai Xu, qiongkai.xu@mq.edu.au, School of Computing

Recommended textbook



Autoregressive Models

- Chain rule based factorization is fully general
- Compact representation via conditional independence and/or neural parameterizations
- **Pros:**
  - Easy to evaluate likelihoods
  - Easy to train
- **Cons:**
  - Requires an ordering
  - Generation is sequential
  - Cannot learn features in an unsupervised way

Plan for Today

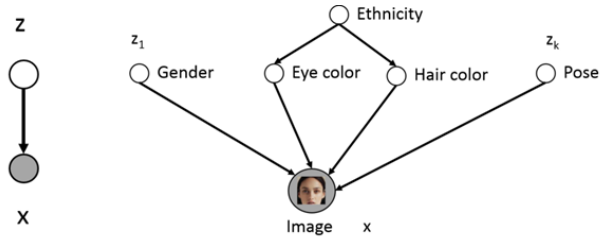
- Latent Variable Models
- Mixture Models/Principal Component Analysis
- Variational Autoencoder
- Variational Inference and Learning

Latent Variable Models: Motivation



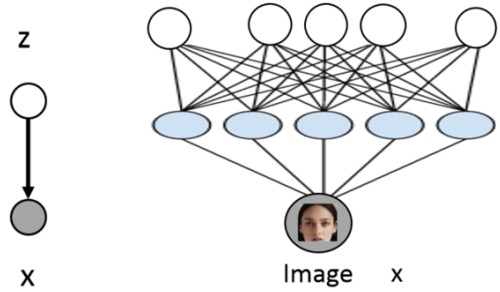
- Lots of variability in images **x** due to gender, eye color, hair color, pose, etc.
- However, unless images are annotated, these factors of variation are not explicitly available (latent).
- **Idea:** Explicitly model these factors using latent variables **z**

Latent Variable Models: Motivation



- Only shared variables $\mathbf{x}$ are observed in the data (pixel values)
- Latent variables $\mathbf{z}$ correspond to high level features
  - if $\mathbf{z}$ chosen properly, $p(\mathbf{x}|\mathbf{z})$ could be much simpler than $p(\mathbf{x})$
  - if we had trained this model, then we could identify features via $p(\mathbf{z}|\mathbf{x})$, e.g., $p(\text{EyeColor} = \text{Blue}|\mathbf{x})$
  - **Challenge**: very difficult to specify these conditionals by hand

Deep Latent Variable Models (Deep LVM)



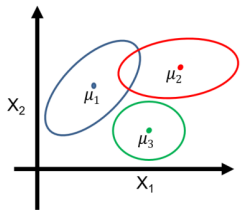
- Use neural networks to model the conditionals (deep latent variable models):

$$\mathbf{z} \sim \mathcal{N}(0, I), p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$$

  where $\mu_\theta, \Sigma_\theta$ are neural networks
- Hope that after training, $\mathbf{z}$ will correspond to meaningful latent factors of variation (features). Unsupervised representation learning.

Mixture of Gaussians. Bayes net: $\mathbf{z} \to \mathbf{x}$

$$\mathbf{z} \sim \text{Categorical}(1, \ldots, K), p(\mathbf{x} \mid \mathbf{z} = k) = \mathcal{N}(\mu_k, \Sigma_k)$$

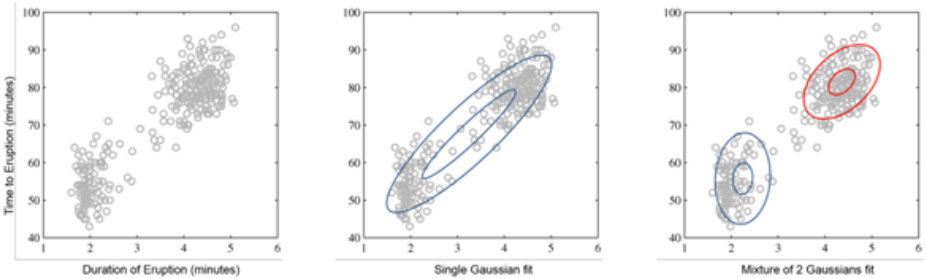


Generative Process:

1. Pick a mixture component k by sampling $\mathbf{z}$
2. Generate a data point by sampling from that Gaussian

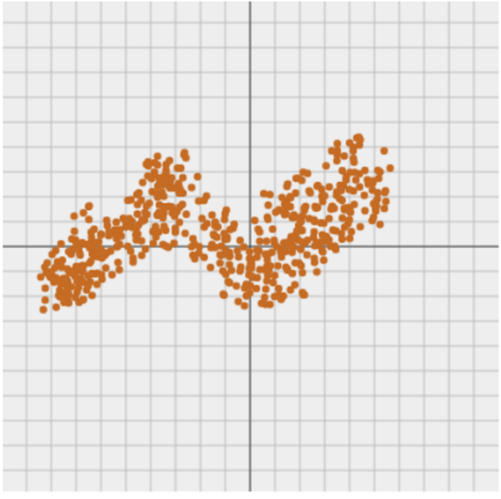
Mixture of Gaussians:

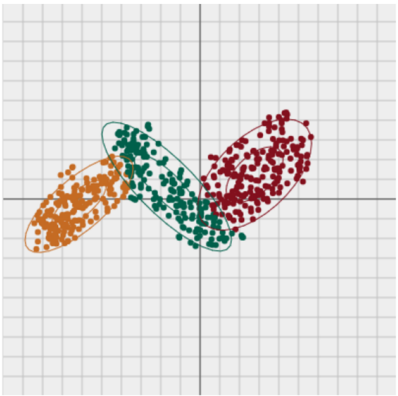
$$\mathbf{z} \sim \text{Categorical}(1, \ldots, K), p(\mathbf{x} \mid \mathbf{z} = k) = \mathcal{N}(\mu_k, \Sigma_k)$$



- **Clustering:** The posterior $p(\mathbf{z} \mid \mathbf{x})$ identifies the mixture component
- **Unsupervised learning:** We are hoping to learn from unlabeled data

Unsupervised Learning





- Show in the posterior probability that a data point was generated by the i-th mixture component, $P(\mathbf{z} = i \mid \mathbf{x})$

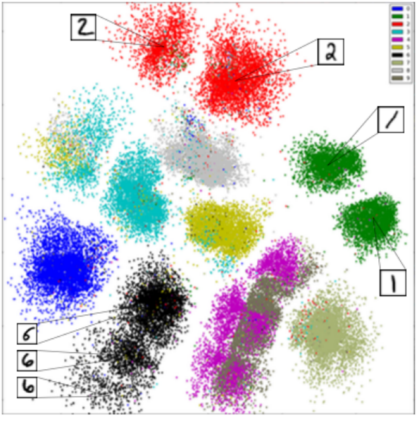
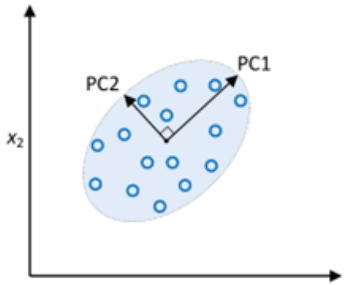


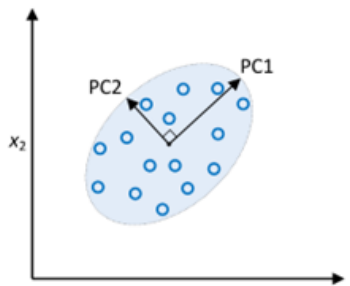
Figure 1: Unsupervised clustering of handwritten digits.

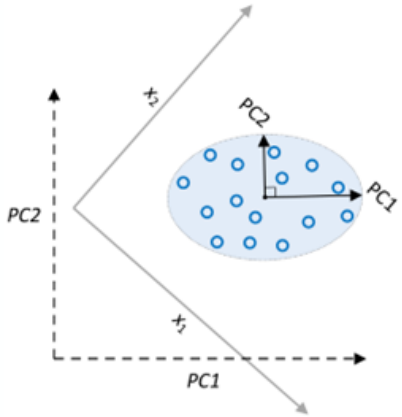
Principle Component Analysis

- 1) Find directions of maximum variance

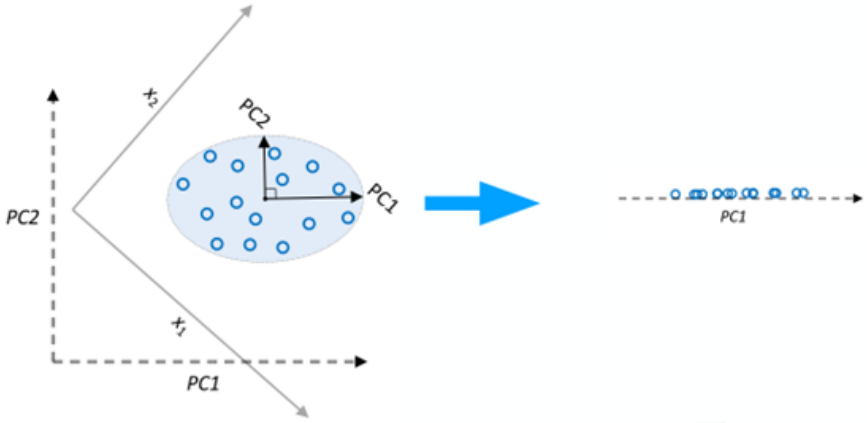


- 2) Transform features into directions of maximum variance

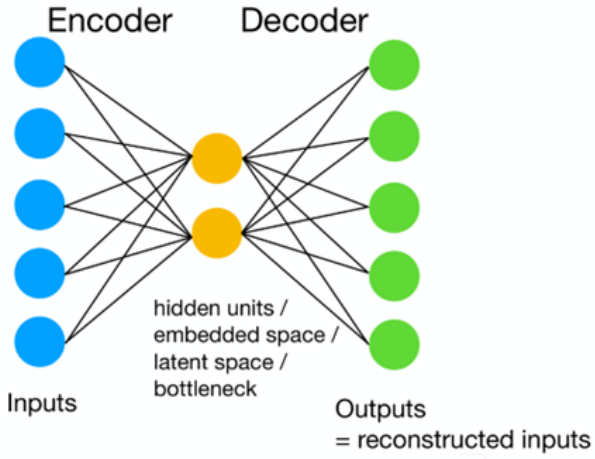




- 3) Usually consider a subset of vectors of most variance (dimensionality reduction)

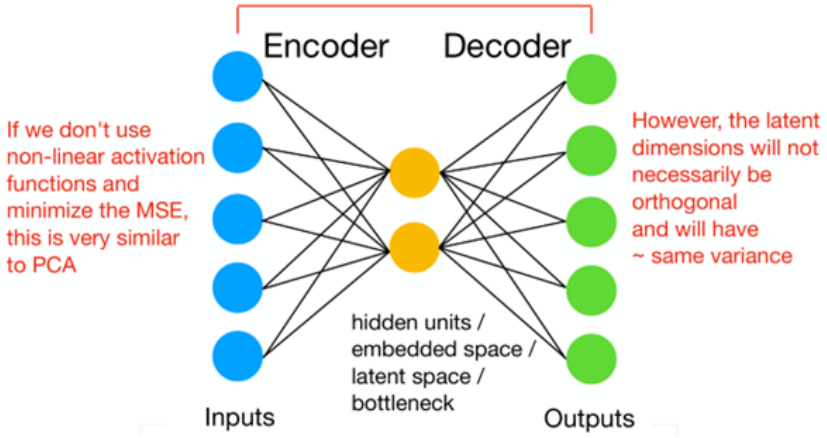


A basic fully-connected (multiplayer perceptron) autoencoder

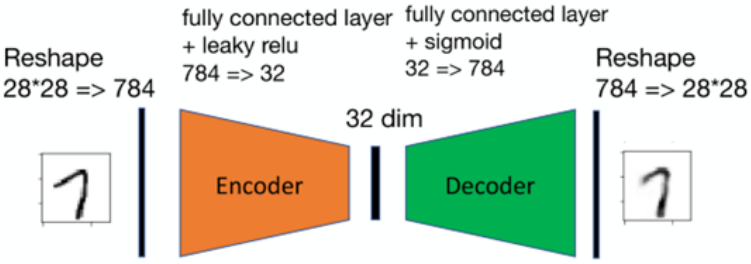


A basic fully-connected (multiplayer perceptron) autoencoder

$$\mathcal{L}(\mathbf{x}, \mathbf{x}') = ||\mathbf{x} - \mathbf{x}'||_2^2 = \sum_i (x_i - x_i')^2$$

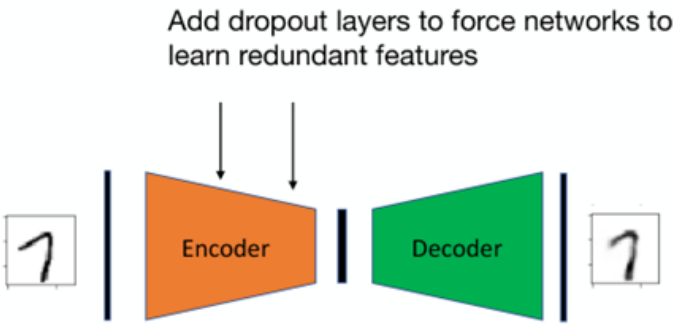


A Simple Autoencoder



Other types of autoencoder

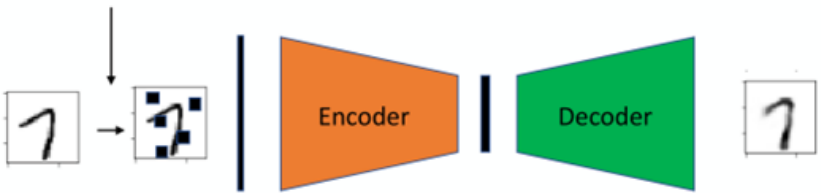
- Autoencoders and Dropout



Other types of autoencoder

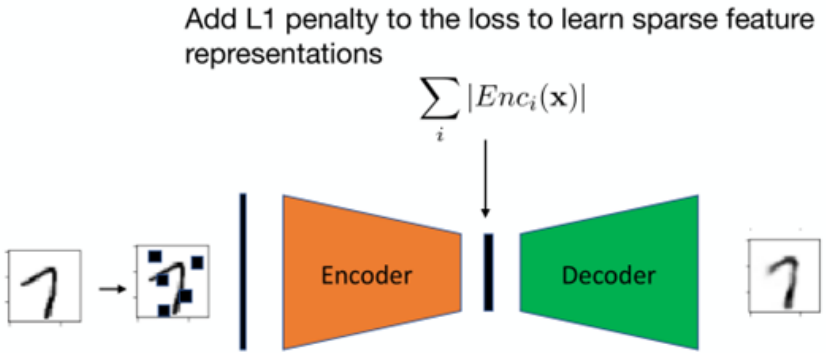
- Denoising Autoencoder

Add dropout after the input, or add noise to the input to learn to denoise images



Other types of autoencoder

- Sparse autoencoder



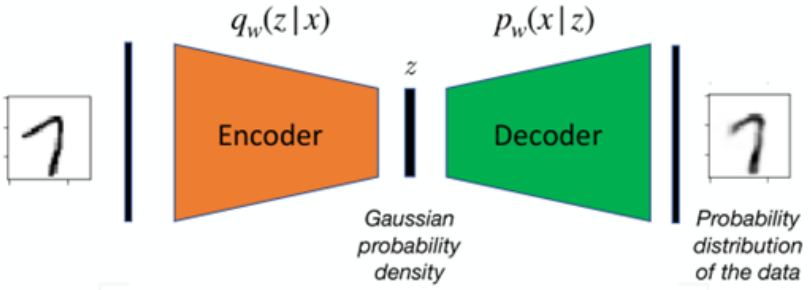
$$\mathcal{L} = ||\mathbf{x} - Dec(Enc(\mathbf{x}))||_2^2 + \sum_i |Enc_i(\mathbf{x})|$$

- Variational Autoencoder

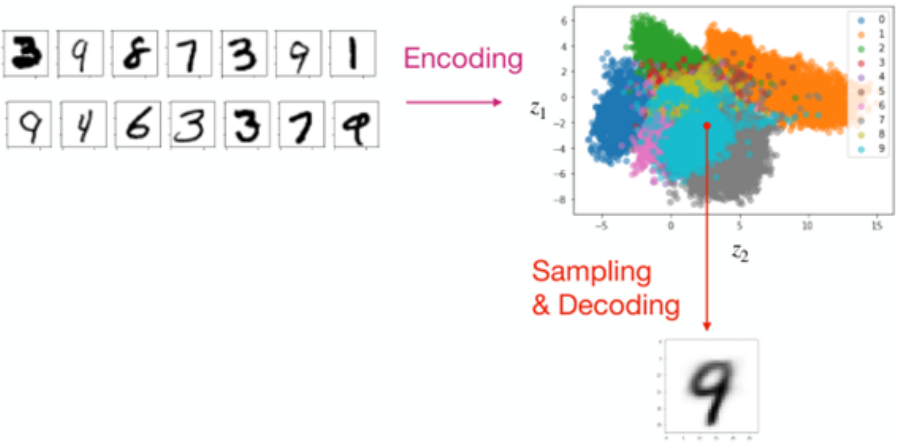
$$L^{[i]} = -\mathbb{E}_{z\sim q_w(z|x^{[i]})}\left[\log p_w\left(x^{[i]}|z\right)\right] + \mathbf{KL}\left(q_w\left(z|x^{[i]}\right)\|p(z)\right)$$

Expected neg. log likelihood term; wrt to encoder distribution

Kullback-Leibler divergence term where $p(z) = \mathcal{N}\left(\mu = 0, \sigma^2 = 1\right)$



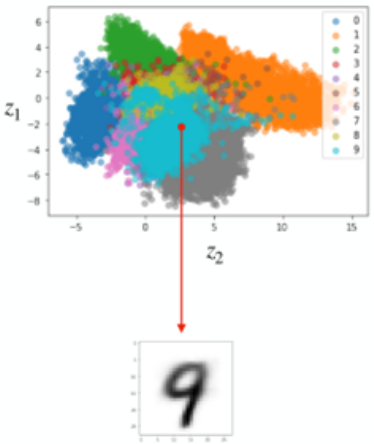
Using regular encoder for sampling



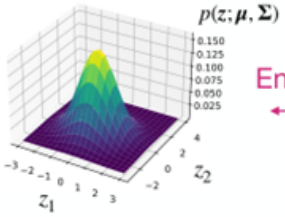
Using regular encoder for sampling

Challenge: regular autoencoders are difficult to sample from, because

1. oddly shaped distribution, hard to sample in a balanced way
2. distribution not centered at (0, 0)
3. distribution not necessarily continuous
   (hard to see here in 2D, but a big problem in higher dimensional latent spaces)



Using variational autoencoder for sampling



Encoding



d-dimensional probability density for multivariate Gaussian

$$p(z;\mu,\Sigma) = \frac{1}{\sqrt{(2\pi)^n |\Sigma|}} \exp\left(-\frac{1}{2}(z-\mu)^T \Sigma^{-1}(z-\mu)\right)$$

$$Z \sim \mathcal{N}(0, I)$$

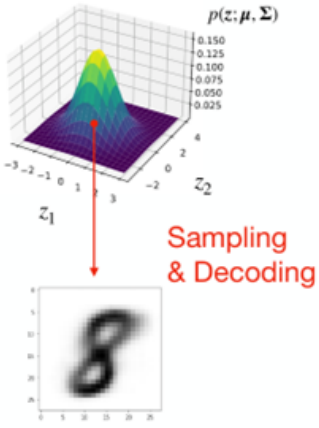
with $z = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix}, \mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$

Sampling from a VAE

$$z = \mu + \sigma \cdot \epsilon$$

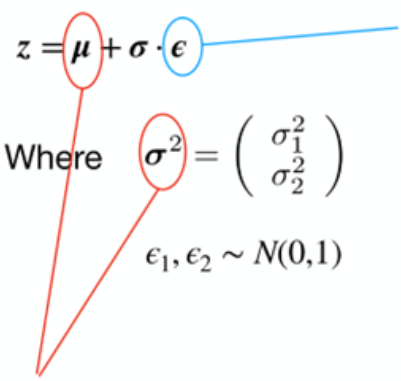
Where $\sigma^2 = \begin{pmatrix} \sigma_1^2 \\ \sigma_2^2 \end{pmatrix}$

$$\epsilon_1, \epsilon_2 \sim N(0,1)$$



- VAE's assume a diagonal covariance matrix (no interaction between the features).
- Thus, we only need a mean and a variance vector, no covariance matrix

Sampling from a VAE



Sampled from standard multivariate normal distribution in each forward pass

But why ϵ? Continuous distribution; VAE must ensure that points in neighborhood encode the same image so that when decoding they produce the same image

Think of these as parameter vectors included in training & backpropagation

Sampling from a VAE - the Log-Var trick

Instead of using a variance vector, $\boldsymbol{\sigma}^2 = \begin{pmatrix} \sigma_1^2 \\ \sigma_2^2 \end{pmatrix}$

we use the
log-var vector
to allow for positive and negative values: $\log(\boldsymbol{\sigma}^2)$

Why can we do this?

$$\log(\sigma^2) = 2 \cdot \log(\sigma)$$
$$\log(\sigma^2)/2 = \log(\sigma)$$
$$\sigma = e^{\log(\sigma^2)/2}$$

So, when we sample the points, we can do

$$z = \mu + e^{\log(\sigma^2)/2} \cdot \epsilon$$

Minimizes ELBO (Evidence lower bound), consisting of KL term and reconstruction loss

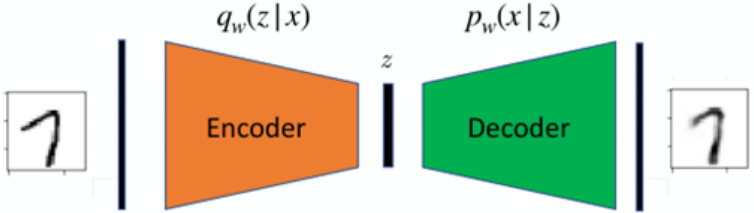
If you assume $p_w(x \mid z)$ follows multivariate-Bernoulli, use cross entropy;
if you assume it follows normal distribution, use MSE

MSE is same as cross-entropy between the empirical distribution and a Gaussian model
(Reference: Deep Learning book by Goodfellow et al., pg. 132)

$$\mathcal{L} = -\mathbb{E}_{z \sim q_w(z \mid x^{[i]})} \left[\log p_w \left(x^{[i]} \mid z \right) \right] + \mathbf{KL} \left(q_w \left(z \mid x^{[i]} \right) \| p(z) \right)$$

Expected neg. log likelihood term; wrt to encoder distribution

Kullback-Leibler divergence term where $p(z) = \mathcal{N}\left(\mu = 0, \sigma^2 = 1\right)$

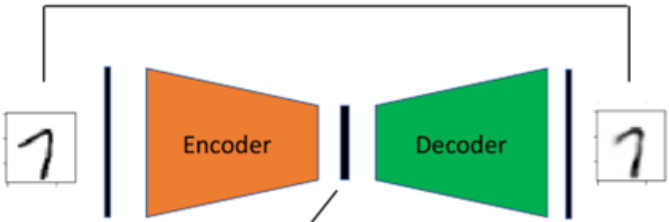


The loss function of VAEs

MACQUARIE University

1) Minimize squared error loss: (ensures good reconstruction)

$$\mathcal{L}_1 = ||\mathbf{x} - Dec(Enc(\mathbf{x}))||_2^2 = \sum_{i=1}^{d} (x_i - x_i')^2$$



2) Minimize KL divergence:

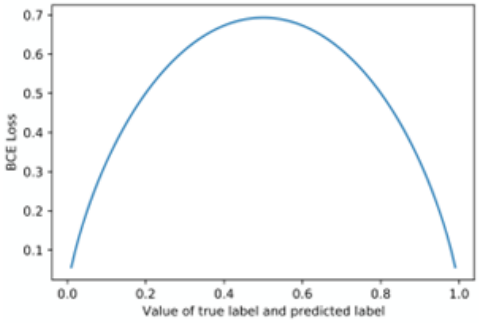
$$\mathcal{L}_2 = D_{KL}\left[N(\mu,\sigma)\|N(0,1)\right] = -\frac{1}{2}\sum\left(1 + \log\left(\sigma^2\right) - \mu^2 - \sigma^2\right)$$

(ensures latent space is continuous and standard normal distributed)

Overall loss: $\mathcal{L} = \alpha \cdot \mathcal{L}_1 + \mathcal{L}_2$

Binary Cross Entropy vs MSE

Cross Entropy is not symmetric:



$$H(p,q) = -\sum_{x \in \mathcal{X}} p(x) \cdot \log q(x)$$

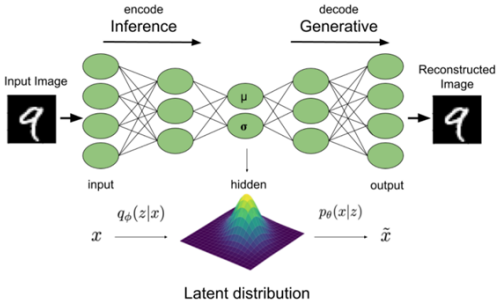
pixel in x

pixel in x'

-0.8 * log(0.7) = 0.285340
-0.8 * log(0.9) = 0.0842884

-0.2 * log(0.1) = 0.460517
-0.2 * log(0.3) = 0.240795

Deriving the ELBO

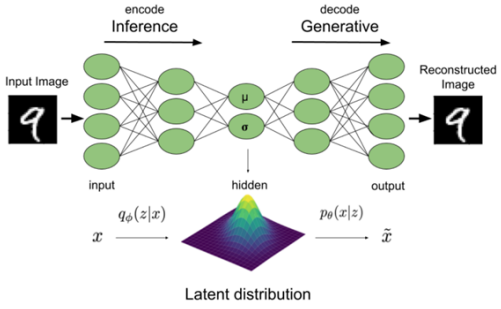


$$\log p(x;\theta) = \log \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\frac{p(\mathbf{z},\mathbf{x};\theta)}{q_\phi(\mathbf{z}\mid\mathbf{x})}\right]$$

$$\geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z},\mathbf{x};\theta) - \log q_\phi(\mathbf{z}\mid\mathbf{x})] := \mathcal{L}(\mathbf{x};\theta,\phi)$$

$\mathcal{L}$ sets a lower bound for the evidence of observations and maximizes $\mathcal{L}$ will push up the log-likelihood of x. Hence, we call $\mathcal{L}$ the evidence lower bound (ELBO).

Autoencoder Perspective



$$\begin{aligned}\mathcal{L}(\mathbf{x};\theta,\phi) &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z},\mathbf{x};\theta) - \log q_\phi(\mathbf{z}\mid\mathbf{x})] \\ &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z},\mathbf{x};\theta) - \log p(\mathbf{z}) + \log p(\mathbf{z}) - \log q_\phi(\mathbf{z}\mid\mathbf{x})] \\ &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}\mid\mathbf{z};\theta)] - D_{KL}(q_\phi(\mathbf{z}\mid\mathbf{x})\|p(\mathbf{z}))\end{aligned}$$

Autoencoder Perspective

- Take a data point $\mathbf{x}^i$, map it to $\hat{\mathbf{z}}$ by sampling from $q_\phi(\mathbf{z} \mid \mathbf{x}^i)$ (encoder). Sample from a Gaussian with parameters $(\mu, \sigma) = \text{encoder}_\phi(\mathbf{x}^i)$
- Reconstruct $\hat{\mathbf{x}}$ by sampling from $p(\mathbf{x} \mid \hat{\mathbf{z}}; \theta)$ (decoder). Sample from a Gaussian with parameters $\text{decoder}_\theta(\hat{\mathbf{z}})$
- What does the training objective $\mathcal{L}(\mathbf{x}; \theta, \phi)$ do?
  - First term encourages $\hat{\mathbf{x}} \approx \mathbf{x}^i$ ($\mathbf{x}^i$ likely under $p(\mathbf{x} \mid \hat{\mathbf{z}}; \theta)$). Autoencoding loss!
  - Second term encourages $\hat{\mathbf{z}}$ to have a distribution similar to the prior $p(\mathbf{z})$

Autoencoder Perspective



- Alice goes on a space mission and needs to send images to Bob. Given an image $\mathbf{x}^i$, she (stochastically) compresses it using $\hat{\mathbf{z}} \sim q_\phi(\mathbf{z} \mid \mathbf{x}^i)$ obtaining a message $\hat{\mathbf{z}}$. Alice sends the message $\hat{\mathbf{z}}$ to Bob
- Given $\hat{\mathbf{z}}$, Bob tries to reconstruct the image using $p(\mathbf{x} \mid \hat{\mathbf{z}}; \theta)$
- This scheme works well if $\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x} \mid \mathbf{z}; \theta)]$ is large
- The term $D_{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$ forces the distribution over messages to have a specific shape $p(\mathbf{z})$. If Bob knows $p(\mathbf{z})$, he can generate realistic messages $\hat{\mathbf{z}} \sim p(\mathbf{z})$ and the corresponding image, as if he had received them from Alice!

VAE in Action!

- We summarize what we have learned:
  - $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_\phi(\mathbf{x}), \sigma^2_\phi(\mathbf{x}))$
  - $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$
  - $p_\theta(\mathbf{x} \mid \mathbf{z}) = \text{Categorical}(\mathbf{x} \mid \theta(\mathbf{z}))$.
- Next, we will use the following networks:
  - The *encoder network*:

$$\mathbf{x} \in \{0, 1, \ldots, 255\}^D \to \text{Linear}(D, 256) \to \text{LeakyReLu}$$
$$\to \text{Linear}(256, 2 \cdot M) \to \text{split} \to \mu \in \mathbb{R}^M, \log\sigma^2 \in \mathbb{R}^M$$

  - The *decoder network*:

$$\mathbf{z} \in \mathbb{R}^M \to \text{Linear}(M, 256) \to \text{LeakyReLu}$$
$$\to \text{Linear}(256, D \cdot L) \to \text{reshape} \to \text{softmax} \to \theta \in [0, 1]^{D \times L}$$

- For a given dataset $\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N$, the training objective is the EBLO where we use a single sample from the variational posterior $\mathbf{z}_{\phi,n} = \mu_\phi(\mathbf{x}_n) + \sigma_\phi(\mathbf{x}_n) \odot \epsilon$, which is:

$$-\text{ELBO}(\mathcal{D}; \boldsymbol{\theta}, \boldsymbol{\phi}) = \sum_{n=1}^{N} - \{\ln \text{Categorical}(\mathbf{x}_n \,|\, \theta(\mathbf{z}_{\phi,n})) - \left[\ln \mathcal{N}(\mathbf{z}_{\phi,n}; \mu_\phi(\mathbf{x}_n), \sigma_\phi^2(\mathbf{x}_n)) - \ln \mathcal{N}(\mathbf{z}_{\phi,n}; \mathbf{0}, \mathbf{I})\right]\} \tag{1}$$

VAE in Action! - continue

1. Take $\mathbf{x}_n$ and apply the encoder network to get $\mu_\phi(\mathbf{x}_n)$ and $\ln \sigma^2_\phi(\mathbf{x}_n)$.
2. Calculate $\mathbf{z}_{\phi,n}$ by applying the reparameterization trick, $\mathbf{z}_{\phi,n} = \mu_\phi(\mathbf{x}_n) + \sigma_\phi(\mathbf{x}_n) \odot \epsilon$, where $\epsilon \sim \mathcal{N}(\epsilon; 0, \mathbf{I})$.
3. Apply the decoder network to $\mathbf{z}_{\phi,n}$ to get the probabilities $\theta(\mathbf{z}_{\phi,n})$
4. Calculate the ELBO by plugging in $\mathbf{x}_n, \mathbf{z}_{\phi,n}, \mu_\phi(\mathbf{x}_n)$ and $\ln \sigma^2_\phi(\mathbf{x}_n)$.

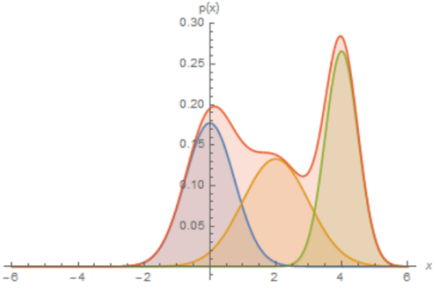
Summary of Latent Variable Models

MACQUARIE University

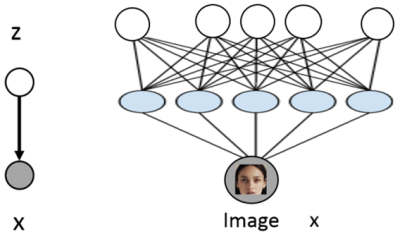
- Combine simple models to get a more flexible one (e.g., mixture of Gaussians)
- Directed model permits ancestral sampling (efficient generation): $\mathbf{z} \sim p(\mathbf{z}), \mathbf{x} \sim p(\mathbf{x} \mid \mathbf{z}; \theta)$
- However, log-likelihood is generally intractable, hence learning is difficult
- Joint learning of a model (θ) and an amortized inference component (ϕ) to achieve tractability via ELBO optimization
- Latent representations for any $\mathbf{x}$ can be inferred via $q_\phi(\mathbf{z} \mid \mathbf{x})$

Thank you!

Alternative Motivation: Combine simple models into a more complex and expressive one



$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) = \sum_{\mathbf{z}} p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) = \sum_{k=1}^{K} p(\mathbf{z} = k) \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$$



A mixture of an infinite number of Gaussians:

$$\mathbf{z} \sim \mathcal{N}(0, I), p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$$

where $\mu_\theta, \Sigma_\theta$ are neural networks

$$\mu_\theta(\mathbf{z}) = (\sigma(a_1\mathbf{z} + c_1), \sigma(a_2\mathbf{z} + c_2)), \Sigma_\theta(\mathbf{z}) = \begin{pmatrix} \exp(\sigma(a_1\mathbf{z} + d_1)) & 0 \\ 0 & \exp(\sigma(a_2\mathbf{z} + d_2)) \end{pmatrix}$$

$$\theta = (A, B, c, d)$$

MACQUARIE University

Likelihood function $p_\theta(\mathbf{x})$ for Partially Observed Data is hard to compute:

$$p_\theta(\mathbf{x}) = \sum_{\text{All values of } \mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) = |\mathcal{Z}| \sum_{\mathbf{z} \in \mathcal{Z}} \frac{1}{|\mathcal{Z}|} p_\theta(\mathbf{x}, \mathbf{z}) = |\mathcal{Z}| \mathbb{E}_{\mathbf{z} \sim \text{Uniform}(\mathcal{Z})}[p_\theta(\mathbf{x}, \mathbf{z})]$$

It is an (intractable) expectation. Monte Carlo to the rescue:

- Sample $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(k)}$ uniformly at random
- Approximate expectation: average $\sum_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) \approx |\mathcal{Z}| \frac{1}{k} \sum_{j=1}^{k} p_\theta(\mathbf{x}, \mathbf{z}^{(j)})$
- Works in theory but not in practice. For most $\mathbf{z}$, $p_\theta(\mathbf{x}, \mathbf{z})$ is very low (most completions don't make sense). Some completions have large $p_\theta(\mathbf{x}, \mathbf{z})$ but we will never "hit" likely completions by uniform random sampling. Need a clever way to select $\mathbf{z}^{(j)}$ to reduce variance of the estimator.

- **Latent Variable Models**
- Allow us to define complex models $p(\mathbf{x})$ in terms of simpler building blocks $p(\mathbf{x} \mid \mathbf{z})$
- Natural for unsupervised learning tasks (clustering, unsupervised representation learning, etc.)
- No free lunch: much more difficult to learn compared to fully observed, autoregressive models

Marginal Likelihood



- Suppose some pixel values are missing at train time (e.g., top half)
- $\mathbf{X}$: observed random variables, $\mathbf{Z}$: unobserved ones (hidden or latent)
- Suppose we have a joint distribution (e.g., PixelCNN) $p(\mathbf{X}, \mathbf{Z}; \theta)$
- What is the probability $p(\mathbf{X} = \mathbf{x}; \theta)$ of observing a training data point $\mathbf{x}$?

$$p(\mathbf{X} = \mathbf{x}; \theta) = \sum_{\mathbf{z}} p(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}; \theta) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta)$$

- Need to consider all possible ways to complete the image (fill green part)

A mixture of an infinite number of Gaussians:

$$\mathbf{z} \sim \mathcal{N}(0, I), p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$$

where $\mu_\theta, \Sigma_\theta$ are neural networks

- $\mathbf{Z}$ are unobserved at train time (also called hidden or latent)
- Suppose we have a model for the joint distribution. What is the probability $p(\mathbf{X} = \mathbf{x}; \theta)$ of observing a training data point $\mathbf{x}$?

$$\int_{\mathbf{z}} p(\mathbf{X} = \mathbf{x}, \mathbf{z} = \mathbf{z}; \theta) d\mathbf{z} = \int_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta) d\mathbf{z}$$



Partially Observed Data

MACQUARIE University

- Suppose that our joint distribution is $p(\mathbf{X}, \mathbf{Z}; \theta)$
- We have a dataset $\mathcal{D}$, where for each datapoint the $\mathbf{X}$ variables are observed (e.g., pixel values) and the variables $\mathbf{Z}$ are never observed (e.g., cluster or class id.). $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(M)}\}$.
- **Maximum likelihood learning:**

$$\log \prod_{\mathbf{x} \in \mathcal{D}} p(\mathbf{x}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log p(\mathbf{x}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta)$$

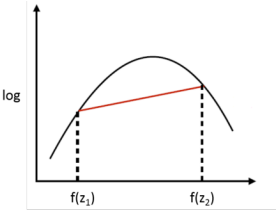
- Evaluating $\log \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta)$ can be intractable. Suppose we have 30 binary latent features, $\mathbf{z} \in \{0, 1\}^{30}$. Evaluating $\sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta)$ involves a sum with 2^{30} terms. For continuous variables, $\log \int_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}; \theta) d\mathbf{z}$ is often intractable. Gradients ∇_θ also hard to compute.
- Need approximations. One gradient evaluation per training data point $\mathbf{x} \in \mathcal{D}$, so approximation needs to be cheap.

Log-Likelihood function for Partially Observed Data is hard to compute:

$$\log\left(\sum_{\mathbf{z}\in\mathcal{Z}} p_\theta(\mathbf{x},\mathbf{z})\right) = \log\left(\sum_{\mathbf{z}\in\mathcal{Z}} \frac{q(\mathbf{z})}{q(\mathbf{z})} p_\theta(\mathbf{x},\mathbf{z})\right) = \log\left(\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}\left[\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right]\right)$$

- Idea: use Jensen Inequality

$$\log\left(\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}[f(\mathbf{z})]\right) = \log\left(\sum_{\mathbf{z}} q(\mathbf{z}) f(\mathbf{z})\right) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log f(\mathbf{z}) = \mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}[\log f(\mathbf{z})]$$



MACQUARIE University

Log-Likelihood function for Partially Observed Data is hard to compute:

$$\log\left(\sum_{\mathbf{z}\in\mathcal{Z}} p_\theta(\mathbf{x},\mathbf{z})\right) = \log\left(\sum_{\mathbf{z}\in\mathcal{Z}} \frac{q(\mathbf{z})}{q(\mathbf{z})} p_\theta(\mathbf{x},\mathbf{z})\right) = \log\left(\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}\left[\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right]\right)$$

- Idea: use Jensen Inequality

$$\log\left(\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}[f(\mathbf{z})]\right) = \log\left(\sum_{\mathbf{z}} q(\mathbf{z}) f(\mathbf{z})\right) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log f(\mathbf{z}) = \mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}[\log f(\mathbf{z})]$$

- Choosing $f(\mathbf{z}) = \frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}$

$$\log\left(\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}\left[\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right]\right) \geq \mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}\left[\log\left(\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right)\right]$$

- Called Evidence Lower Bound (ELBO).

- Suppose $q(\mathbf{z})$ is any probability distribution over the hidden variables
- Evidence Lower Bound (ELBO) holds for any q

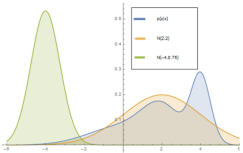
$$\begin{aligned}\log p(\mathbf{x};\theta) &\geq \sum_{\mathbf{z}} q(\mathbf{z}) \log\left(\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right) \\ &= \sum_{\mathbf{z}} q(\mathbf{z}) \log p_\theta(\mathbf{x},\mathbf{z}) - \sum_{\mathbf{z}} q(\mathbf{z}) \log q(\mathbf{z})\end{aligned}$$

- Equality holds if $q = p(\mathbf{z} \mid \mathbf{x};\theta)$

$$\log p(\mathbf{x};\theta) = \sum_{\mathbf{z}} q(\mathbf{z}) \log p_\theta(\mathbf{x},\mathbf{z}) - \sum_{\mathbf{z}} q(\mathbf{z}) \log q(\mathbf{z})$$

- (Aside: This is what we compute in the E-step of the EM algorithm)

The Evidence Lower Bound



- What if the posterior $p(\mathbf{z} \mid \mathbf{x}; \theta)$ is intractable to compute?
- Suppose $q(\mathbf{z}; \phi)$ is a (tractable) probability distribution over the hidden variables parameterized by ϕ (variational parameters)
- For example, a Gaussian with mean and covariance specified by ϕ

$$q(\mathbf{z}; \phi) = \mathcal{N}(\phi_1, \phi_2)$$

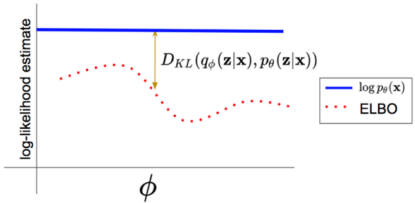
- Variational inference: pick ϕ so that $q(\mathbf{z}; \phi)$ is as close as possible to $p(\mathbf{z} \mid \mathbf{x}; \theta)$. In the figure, the posterior $p(\mathbf{z} \mid \mathbf{x}; \theta)$ (blue) is better approximated by $\mathcal{N}(2, 2)$ (orange) than $\mathcal{N}(-4, 0.75)$ (green)

A Variational Approximation to the Posterior



- Assume $p(\mathbf{x}^{\text{top}}, \mathbf{x}^{\text{bottom}}; \theta)$ assigns high probability to images that look like digits. In this example, we assume $\mathbf{z} = \mathbf{x}^{\text{top}}$ are unobserved (latent)
- Suppose $q(\mathbf{x}^{\text{top}}; \phi)$ is a (tractable) probability distribution over the hidden variables (missing pixels in this example) $\mathbf{x}^{\text{top}}$ parameterized by ϕ (variational parameters) $q(\mathbf{x}^{\text{top}}; \phi) = \prod_{\text{unobserved variables } \mathbf{x}_i^{\text{top}}} (\phi_i)^{\mathbf{x}_i^{\text{top}}} (1 - \phi_i)^{(1-\mathbf{x}_i^{\text{top}})}$
- Is $\phi_i = 0.5 \, \forall i$ a good approximation to the posterior $p(\mathbf{x}^{\text{top}} \mid \mathbf{x}^{\text{bottom}}; \theta)$? No
- Is $\phi_i = 1 \, \forall i$ a good approximation to the posterior $p(\mathbf{x}^{\text{top}} \mid \mathbf{x}^{\text{bottom}}; \theta)$? No
- Is $\phi_i \approx 1$ for pixels i corresponding to the top part of digit 9 a good approximation? Yes

The Evidence Lower Bound



$$\log p(\mathbf{x};\theta) \geq \sum_{\mathbf{z}} q(\mathbf{z};\phi) \log p(\mathbf{z},\mathbf{x};\theta) + H(q(\mathbf{z};\phi)) = \underbrace{\mathcal{L}(\mathbf{x};\theta,\phi)}_{\text{ELBO}} \tag{2}$$

- The better $q(\mathbf{z};\phi)$ can approximate the posterior $p(\mathbf{z} \mid \mathbf{x};\theta)$, the smaller $D_{KL}(q(\mathbf{z};\phi) \| p(\mathbf{z} \mid \mathbf{x};\theta))$ we can achieve, the closer ELBO will be to $\log p(\mathbf{x};\theta)$.
- Next: jointly optimize over θ and ϕ to maximize the ELBO over a dataset

Summary

- **Latent Variable Models Pros:**
  - Easy to build flexible models
  - Suitable for unsupervised learning
- **Latent Variable Models Cons:**
  - Hard to evaluate likelihoods
  - Hard to train via maximum-likelihood
- Fundamentally, the challenge is that posterior inference $p(\mathbf{z} \mid \mathbf{x})$ is hard. Typically requires variational approximations
- Alternative: give up on KL-divergence and likelihood (GANs)

- **Latent Variable Models**
- Allow us to define complex models $p(\mathbf{x})$ in terms of simple building blocks $p(\mathbf{x} \mid \mathbf{z})$
- Natural for unsupervised learning tasks (clustering, unsupervised representation learning, etc.)
- No free lunch: much more difficult to learn compared to fully observed, autoregressive models because $p(\mathbf{x})$ is hard to evaluate (and optimize)

- Suppose $q(\mathbf{z})$ is any probability distribution over the hidden variables
- Evidence Lower Bound (ELBO) holds for any q:

$$\begin{aligned}\log p(\mathbf{x};\theta) &\geq \sum_{\mathbf{z}} q(\mathbf{z}) \log\left(\frac{p_\theta(\mathbf{x},\mathbf{z})}{q(\mathbf{z})}\right) \\ &= \sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{x},\mathbf{z}) - \sum_{\mathbf{z}} q(\mathbf{z}) \log q(\mathbf{z}) \\ &= \sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{x},\mathbf{z}) + H(q)\end{aligned}$$

- Suppose $q(\mathbf{z})$ is any probability distribution over the hidden variables. A little bit of algebra reveals:

$$D_{KL}(q(\mathbf{z})\|p(\mathbf{z} \mid \mathbf{x}; \theta)) = -\sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{z}, \mathbf{x}; \theta) + \log p(\mathbf{x}; \theta) - H(q) \geq 0$$

- Evidence Lower Bound (ELBO) holds for any q:

$$\log p(\mathbf{x}; \theta) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q)$$

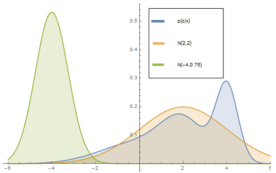
- Equality holds if $q = p(\mathbf{z} \mid \mathbf{x}; \theta)$ because $D_{KL}(q(\mathbf{z})\|p(\mathbf{z} \mid \mathbf{x}; \theta)) = 0$:

$$\log p(\mathbf{x}; \theta) = \sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q)$$

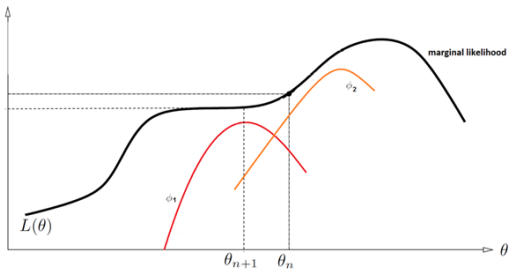
- Confirms our intuition that we seek likely completions $\mathbf{z}$ given the observed values (evidence) $\mathbf{x}$.

Intractable Posteriors

- What if the posterior $p(\mathbf{z} \mid \mathbf{x}; \theta)$ is intractable to compute? In a VAE, this corresponds to "inverting" the neural networks $\mu_\theta, \Sigma_\theta$ defining $p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$
- Suppose $q(\mathbf{z}; \phi)$ is a (tractable) probability distribution over the hidden variables parameterized by ϕ (variational parameters)
- E.g., a Gaussian specified by ϕ $q(\mathbf{z}; \phi) = \mathcal{N}(\phi_1, \phi_2)$



- Variational inference: pick ϕ so that $q(\mathbf{z}; \phi)$ is close to $p(\mathbf{z} \mid \mathbf{x}; \theta)$.
- In the figure, the posterior $p(\mathbf{z} \mid \mathbf{x}; \theta)$ (blue) is better approximated by $\mathcal{N}(2, 2)$ (orange) than $\mathcal{N}(-4, 0.75)$ (green)



- $\mathcal{L}(\mathbf{x}; \theta, \phi_1)$ and $\mathcal{L}(\mathbf{x}; \theta, \phi_2)$ are both lower bounds.
- We want to jointly optimize θ, ϕ

$$\mathcal{L}(\mathbf{x}; \theta, \phi) = \mathbb{E}_{q(\mathbf{z};\phi)}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q(\mathbf{z}; \phi)]$$

- Note: dropped i superscript from ϕ^i for compactness
- To evaluate the bound, sample $\mathbf{z}^1, \ldots, \mathbf{z}^K$ from $q(\mathbf{z}; \phi)$ and estimate:

$$\mathbb{E}_{q(\mathbf{z};\phi)}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q(\mathbf{z}; \phi)] \approx \frac{1}{K} \sum_{k=1}^{K} \log p(\mathbf{z}^k, \mathbf{x}; \theta) - \log q(\mathbf{z}^k; \phi)$$

- Key assumption: $q(\mathbf{z}; \phi)$ is tractable, i.e., easy to sample from and evaluate
- Want to compute $\nabla_\theta \mathcal{L}(\mathbf{x}; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(\mathbf{x}; \theta, \phi)$
- The gradient with respect to θ is easy:

$$\begin{aligned}\nabla_\theta \mathbb{E}_{q(\mathbf{z};\phi)}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q(\mathbf{z}; \phi)] &= \mathbb{E}_{q(\mathbf{z};\phi)}[\nabla_\theta \log p(\mathbf{z}, \mathbf{x}; \theta)] \\ &\approx \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta \log p(\mathbf{z}^k, \mathbf{x}; \theta)\end{aligned}$$

$$\mathcal{L}(\mathbf{x}; \theta, \phi) = \sum_{\mathbf{z}} q(\mathbf{z}; \phi) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q(\mathbf{z}; \phi))$$
$$= \mathbb{E}_{q(\mathbf{z};\phi)}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q(\mathbf{z}; \phi)]$$

- Want to compute $\nabla_\theta \mathcal{L}(\mathbf{x}; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(\mathbf{x}; \theta, \phi)$
- The gradient with respect to ϕ is more complicated because the expectation depends on ϕ
- We still want to estimate with a Monte Carlo average
- Later in the course, we'll see a general technique called REINFORCE (from reinforcement learning)
- For now, a better but less general alternative that only works for continuous $\mathbf{z}$ (and only some distributions)

- Want to compute a gradient with respect to ϕ of:

$$\mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z})] = \int q(\mathbf{z};\phi) r(\mathbf{z}) d\mathbf{z}, \mathbf{z} \text{ is continuous.}$$

- Suppose $q(\mathbf{z};\phi) = \mathcal{N}(\mu, \sigma^2 I)$ is Gaussian with parameters $\phi = (\mu, \sigma)$.
  - Sample $\mathbf{z} \sim q(\mathbf{z};\phi)$
  - Sample $\epsilon \sim \mathcal{N}(0, I), \mathbf{z} = \mu + \sigma\epsilon = g(\epsilon;\phi)$. g is deterministic!

- Using this equivalence we compute the expectation in two ways:

$$\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z};\phi)}[r(\mathbf{z})] = \int q(\mathbf{z};\phi) r(\mathbf{z}) d\mathbf{z} = \mathbb{E}_{\epsilon\sim\mathcal{N}(0,I)}[r(g(\epsilon;\phi))]$$
$$= \int \mathcal{N}(\epsilon;0,I) r(\mu+\sigma\epsilon) d\epsilon$$

$$\nabla_\phi \mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z})] = \nabla_\phi \mathbb{E}_\epsilon[r(g(\epsilon;\phi))] = \mathbb{E}_\epsilon[\nabla_\phi r(g(\epsilon;\phi))]$$

- Easy to estimate via Monte Carlo if r and g are differentiable w.r.t. ϕ and ϵ is easy to sample from (backpropagation):

$$\mathbb{E}_\epsilon[\nabla_\phi r(g(\epsilon;\phi))] \approx \frac{1}{K}\sum_{k=1}^{K} \nabla_\phi r(g(\epsilon^k;\phi)) \quad \text{where } \epsilon^1,\cdots,\epsilon^K \sim \mathcal{N}(0,I)$$

$$\mathcal{L}(\mathbf{x}; \theta, \phi) = \sum_{\mathbf{z}} q(\mathbf{z}; \phi) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q(\mathbf{z}; \phi))$$
$$= \mathbb{E}_{q(\mathbf{z};\phi)}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q(\mathbf{z}; \phi)] = \mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z}, \phi)]$$

- Our case is slightly more complicated because we have $\mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z}, \phi)]$ instead of $\mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z})]$. Term inside the expectation also depends on ϕ.
- Can still use reparameterization. Assume $\mathbf{z} = \mu + \sigma\epsilon = g(\epsilon; \phi)$ like before. Then:

$$\mathbb{E}_{q(\mathbf{z};\phi)}[r(\mathbf{z}, \phi)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)}[r(g(\epsilon; \phi), \phi)]$$
$$\approx \frac{1}{K} \sum_{k=1}^{K} r(g(\epsilon^k; \phi), \phi)$$

- Use chain rule for the gradient.

$$\max_{\theta} \ell(\theta; \mathcal{D}) \geq \max_{\theta, \phi^1, \cdots, \phi^M} \sum_{\mathbf{x}^i \in \mathcal{D}} \mathcal{L}(\mathbf{x}^i; \theta, \phi^i)$$

- So far we have used a set of variational parameters ϕ^i for each data point $\mathbf{x}^i$. Does not scale to large datasets.
- Amortization: Now we learn a single parametric function f_λ that maps each $\mathbf{x}$ to a set of (good) variational parameters. Like doing regression on $\mathbf{x}^i \mapsto \phi^{i,*}$
- For example, if $q(\mathbf{z} \mid \mathbf{x}^i)$ are Gaussians with different means $\mu^1, \cdots, \mu^M$, we learn a single neural network f_λ mapping $\mathbf{x}^i$ to μ^i
- We approximate the posteriors $q(\mathbf{z} \mid \mathbf{x}^i)$ using this distribution $q_\lambda(\mathbf{z} \mid \mathbf{x})$

A Variational Approximation to the Posterior



- Assume $p(\mathbf{z}, \mathbf{x}^i; \theta)$ is close to $p_{\text{data}}(\mathbf{z}, \mathbf{x}^i)$. Suppose $\mathbf{z}$ captures information such as the digit identity (label), style, etc.
- Suppose $q(\mathbf{z}; \phi^i)$ is a (tractable) probability distribution over the hidden variables $\mathbf{z}$ parameterized by ϕ^i
- For each $\mathbf{x}^i$, need to find a good $\phi^{i,*}$ (via optimization, expensive).
- Amortized inference: learn how to map $\mathbf{x}^i$ to a good set of parameters ϕ^i via $q(\mathbf{z}; f_\lambda(\mathbf{x}^i))$. f_λ learns how to solve the optimization problem for you
- In the literature, $q(\mathbf{z}; f_\lambda(\mathbf{x}^i))$ often denoted $q_\phi(\mathbf{z} \mid \mathbf{x})$

- Optimize $\sum_{\mathbf{x}^i \in \mathcal{D}} \mathcal{L}(\mathbf{x}^i; \theta, \phi)$ as a function of θ, ϕ using (stochastic) gradient descent:

$$\begin{aligned}\mathcal{L}(\mathbf{x}; \theta, \phi) &= \sum_{\mathbf{z}} q_\phi(\mathbf{z} \mid \mathbf{x}) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q_\phi(\mathbf{z} \mid \mathbf{x})) \\ &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}, \mathbf{x}; \theta) - \log q_\phi(\mathbf{z} \mid \mathbf{x})]\end{aligned}$$

- Initialize $\theta^{(0)}, \phi^{(0)}$
- Randomly sample a data point $\mathbf{x}^i$ from $\mathcal{D}$
- Compute $\nabla_\theta \mathcal{L}(\mathbf{x}^i; \theta, \phi)$ and $\nabla_\phi \mathcal{L}(\mathbf{x}^i; \theta, \phi)$
- Update θ, ϕ in the gradient direction
- How to compute the gradients? Use reparameterization like before

Estimating Log-Likelihoods (optional, skip)

MACQUARIE University

Likelihood function $p_\theta(\mathbf{x})$ for Partially Observed Data is hard to compute:

$$p_\theta(\mathbf{x}) = \sum_{\text{All possible values of } \mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) = \sum_{\mathbf{z} \in \mathcal{Z}} \frac{q(\mathbf{z})}{q(\mathbf{z})} p_\theta(\mathbf{x}, \mathbf{z}) = \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \right]$$

Monte Carlo to the rescue:

- Sample $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(k)}$ from $q(\mathbf{z})$
- Approximate expectation with sample average $p_\theta(\mathbf{x}) \approx \frac{1}{k} \sum_{j=1}^{k} \frac{p_\theta(\mathbf{x}, \mathbf{z}^{(j)})}{q(\mathbf{z}^{(j)})}$
- Recall that for training, we need the log-likelihood $\log(p_\theta(\mathbf{x}))$:

$$\log(p_\theta(\mathbf{x})) \approx \log \left(\frac{1}{k} \sum_{j=1}^{k} \frac{p_\theta(\mathbf{x}, \mathbf{z}^{(j)})}{q(\mathbf{z}^{(j)})} \right) \approx \log \left(\frac{p_\theta(\mathbf{x}, \mathbf{z}^{(1)})}{q(\mathbf{z}^{(1)})} \right)$$

- However, it's clear that $\mathbb{E}_{\mathbf{z}^{(1)} \sim q(\mathbf{z})} \left[\log \left(\frac{p_\theta(\mathbf{x}, \mathbf{z}^{(1)})}{q(\mathbf{z}^{(1)})} \right) \right] \neq \log \left(\mathbb{E}_{\mathbf{z}^{(1)} \sim q(\mathbf{z})} \left[\frac{p_\theta(\mathbf{x}, \mathbf{z}^{(1)})}{q(\mathbf{z}^{(1)})} \right] \right)$

Why is the Bound Tight (optional, skip)

MACQUARIE University

- We derived this lower bound that holds for any choice of $q(\mathbf{z})$:

$$\log p(\mathbf{x};\theta) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log \frac{p(\mathbf{x},\mathbf{z};\theta)}{q(\mathbf{z})}$$

- If $q(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{x};\theta)$ the bound becomes:

$$\begin{aligned}
\sum_{\mathbf{z}} p(\mathbf{z} \mid \mathbf{x};\theta) \log \frac{p(\mathbf{x},\mathbf{z};\theta)}{p(\mathbf{z} \mid \mathbf{x};\theta)} &= \sum_{\mathbf{z}} p(\mathbf{z} \mid \mathbf{x};\theta) \log \frac{p(\mathbf{z} \mid \mathbf{x};\theta)p(\mathbf{x};\theta)}{p(\mathbf{z} \mid \mathbf{x};\theta)} \\
&= \sum_{\mathbf{z}} p(\mathbf{z} \mid \mathbf{x};\theta) \log p(\mathbf{x};\theta) \\
&= \log p(\mathbf{x};\theta) \sum_{\mathbf{z}} p(\mathbf{z} \mid \mathbf{x};\theta) \\
&= \log p(\mathbf{x};\theta)
\end{aligned}$$

Variational Inference Continued (optional, skip)

MACQUARIE University

- Suppose $q(\mathbf{z})$ is any probability distribution over the hidden variables. A little bit of algebra reveals

$$D_{KL}(q(\mathbf{z})\|p(\mathbf{z} \mid \mathbf{x}; \theta)) = -\sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{z}, \mathbf{x}; \theta) + \log p(\mathbf{x}; \theta) - H(q) \geq 0$$

- Rearranging, we re-derived the Evidence Lower Bound (ELBO)

$$\log p(\mathbf{x}; \theta) \geq \sum_{\mathbf{z}} q(\mathbf{z}) \log p(\mathbf{z}, \mathbf{x}; \theta) + H(q)$$

- In general, $\log p(\mathbf{x}; \theta) = \text{ELBO} + D_{KL}(q(\mathbf{z})\|p(\mathbf{z} \mid \mathbf{x}; \theta))$. The closer $q(\mathbf{z})$ is to $p(\mathbf{z} \mid \mathbf{x}; \theta)$, the closer the ELBO is to the true log-likelihood